Cover Page

Field	Detail
Course	CSE 3202 — Artificial Intelligence Sessional
Project Title	Apex Rush: F1 Racing
Team Members (Name, ID)	1. Fahamida Akther Moina, 0802410105101003 2. Mahadi Inthesir Mukit, 0802410205101044
Submission Date	13 September 2026
Instructor	Md Zahim Hassan, Lecturer CSE Md. Minhazul Islam, Lecturer CSE

1. Abstract

Apex Rush: F1 Racing is an interactive 2D/3D Formula 1 racing game where a human player competes against autonomous AI-controlled drivers across 4 to 5 distinct racing tracks. Unlike static rule-based non-player characters (NPCs), the AI agents in Apex Rush are designed to mimic human driving behavior by dynamically controlling acceleration, steering, and braking while making realistic human-like mistakes such as late braking or taking wide turns. The agent utilizes search/planning algorithms combined with stochastic decision-making to balance competitive performance with human-like variability. This ensures balanced winning chances for both the human driver and AI competitors without giving unfair advantage to either party.

2. Introduction & Motivation

Traditional racing games often suffer from AI competitors that either operate on perfect, rigid trajectories (making them predictable and robotic) or rely on artificial speed boosts ("rubber-banding") to stay competitive. Creating believable, human-like AI drivers requires dynamic decision-making that accounts for track geometry, opponent positions, and risk evaluation under uncertainty.

Apex Rush: F1 Racing addresses this challenge by implementing an intelligent agent architecture that models human-like driving performance. The game features 4 to 5 distinct tracks with varying turn complexities and layouts. One car is directly controlled by the human user via input controls, while remaining opponent cars are managed by AI agents that continuously evaluate racing lines and simulate realistic human errors. The intended users are casual gamers and AI enthusiasts interested in evaluating autonomous agent behaviors in real-time simulated environments.

3. Problem Statement & Objectives

Problem Statement: Standard racing game NPCs rely on deterministic path-following or artificial cheating mechanisms, resulting in unrealistic competition that lacks human-like unpredictability, mistakes, and fair win dynamics.

Objectives:

3.1. Design an autonomous driving agent capable of navigating complex race tracks across 4–5 unique map environments.

3.2. Implement a realistic behavior and decision-making model that introduces human-like driver mistakes (e.g., occasional late braking, imperfect racing line recovery) without compromising competitiveness.

3.3. Develop an intuitive racing interface where a human player competes seamlessly against AI-driven vehicles with balanced win probabilities.

3.4. Evaluate the agent's lap performance, mistake frequency, and win-rate balance across multiple test sessions.

4. PEAS Specification

Element	Specification
Performance Measure	Lap completion time, obstacle/wall avoidance accuracy, fair win-loss ratio (~50% competitive balance between human and AI), and human-like mistake frequency without catastrophic game-breaking behavior.
Environment	A simulated 2D/3D racing environment featuring 4–5 distinct race tracks with defined boundaries, turn curvatures, checkpoints, and multi-vehicle physics interactions.
Actuators	Motor controls for acceleration, braking, gear adjustments, and steering angle updates.
Sensors	Virtual raycast sensors/distance sensors (measuring track boundaries and nearby opponent positions), current velocity meter, track waypoint position, and steering angle sensor.

Environment Properties: Continuous (position and velocity change smoothly over time); partially observable (the AI perceives track boundaries and nearby cars within sensor range); multi-agent (human vs. AI agents competing); dynamic (vehicle positions change continuously); sequential (driving actions directly affect future vehicle state and trajectory).

5. Proposed Agent Architecture

Apex Rush utilizes a Model-Based Goal-Based / Utility-Based Agent architecture:

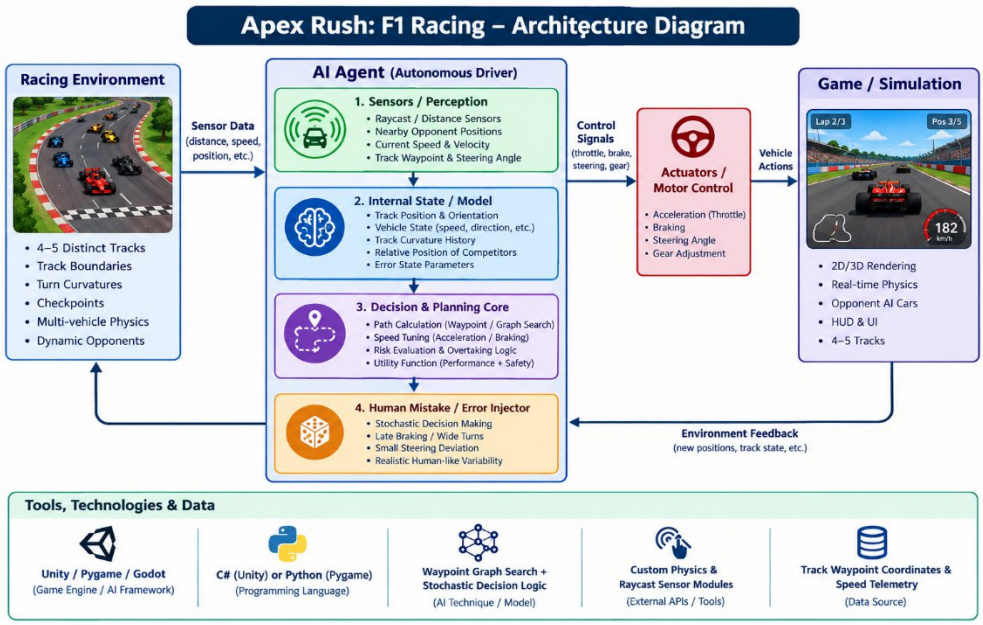
- **Model-Based:** The agent maintains an internal state of its current speed, orientation, track position, and surrounding vehicle distances.

- **Goal-Based & Utility-Based:** The agent's primary goal is to complete track laps in optimal time while maximizing a utility function that balances speed, track boundary safety, and controlled human-like error injection (probabilistic variation in braking/steering actions).

Simple reflex agents are insufficient for this task because taking a turn requires anticipating upcoming track curvature and opponent locations based on velocity, rather than reacting solely to immediate distance sensors.

Architecture Diagram:

Racing Environment
↓
Sensors / Perception
↓
Internal State Model
↓
AI Decision & Planning
↓
Human-Like Behavior
↓
Vehicle Control
↓
Vehicle Movement
↓
Racing Environment



Internal State / Memory: The agent stores current waypoint targets, track curvature history, relative position to competitors, and current error state parameters during each race turn.

6. Tools, Technologies & Data

Category	Choice	Justification
LLM / AI Model	Pygame	Provides robust physics simulation, raycasting sensors, and rendering capabilities.
Framework / SDK	Python (for Pygame)	Standard language for game physics and AI decision logic implementation.
Programming Language	Waypoint Graph Search + Stochastic Decision Logic / Q-Learning	Enables optimal path navigation while allowing probabilistic error injection for realism.

Category	Choice	Justification
External APIs / Tools	Custom Physics & Raycast Sensor Modules	Used for distance sensing and collision detection.
Data Source(s)	Track Waypoint Coordinates & Speed Telemetry	Pre-designed map node data for 4–5 distinct track layouts.

7. Methodology & Implementation Plan

- **Design Phase:** Finalize PEAS parameters, design vector maps for 4–5 distinct tracks, and define vehicle physics equations (speed, acceleration, turning radius).
- **Core Implementation:** Implement vehicle motor controls and build the waypoint-following pathing system for the AI driver.
- **AI & Error Injector Integration:** Develop the sensor raycasting logic and introduce the stochastic error module (randomized sub-optimal braking/steering adjustments) to mirror human driving imperfections.
- **Testing & Refinement:** Conduct test races across all tracks to tune AI speed parameters, ensuring equal win probability and natural-looking opponent behavior.